

Report - Andrea Delli (s331998)

≡ Autore

Andrea Delli

CI2024 Project – Symbolic Regression

This section contains the project report for “CI2024 Project – Symbolic Regression”. This project was developed as the final assignment for the *Computational Intelligence* course at **Politecnico di Torino**, in collaboration with my colleague [Giorgia Modi](#).

Problem Statement

Symbolic regression is the task of discovering a mathematical expression that best fits a given dataset without assuming any fixed parametric form in advance.

Given samples of inputs

$x \in \mathbb{R}^n$ and outputs $y \in \mathbb{R}$, our goal is to automatically find an analytic function $f(x)$ such that $f(x_i) \approx y_i$ with minimal error.

Methodology

1. Representation of Individuals

Each candidate function is an expression and is represented as a tree of nodes. Each node can be:

- **Primitives:** operators/functions such as **add**, **sub**, **mul**, **div**, **pow**, **sin**, **cos**, **exp**, **log**, **abs**, etc. These are intermediate nodes of the tree.
- **Terminals:** input variables $x[0], \dots, x[n-1]$, constants drawn from a uniform pool in $[-50, 50]$, special constants (π , e), integers, and powers of 10. These are leaf nodes of the tree.
Note: during developing, we tried different ranges of constant pools.

2. Initialization

We generate an initial population of size P by sampling random trees up to a **maximum depth** D .

It's possible to restart from a previous saved solution by providing the correct path of the `.pkl` file (using the `solution_path` argument). In this way the genetic algorithm can be interrupted and restored in multiple runs.

3. Fitness Evaluation

As fitness function we use the **Mean Squared Error (MSE)** between predicted $\hat{y}$ and true y :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Invalid expressions (shape mismatch, NaNs, exceptions) are assigned infinite fitness.

4. Parent Selection

We experimented with multiple strategies of parent selection:

- **Tournament Selection:** This method selects a random subset of the population (default size of 10) and chooses the two individuals with the best fitness (lowest error) from within it. This introduces strong selective pressure, promoting rapid convergence toward good solutions. The higher the tournament size, the more likely it is that strong individuals dominate the population early.
- **Rank Selection:** individuals are ranked by fitness, and selection probabilities are assigned based on their rank, with better-ranked individuals having a higher chance of being selected. This softens the pressure compared to tournament selection and helps maintain genetic diversity by giving lower-performing individuals a non-zero chance of selection.

- **Fitness-Proportional (Roulette Wheel):** Also known as "roulette wheel" selection, this approach assigns selection probabilities proportional to the inverse of the fitness score ($1/(\text{fitness} + \varepsilon)$). This rewards individuals with lower mean squared error (MSE) by giving them a higher probability of being selected, but still allows for occasional sampling of less fit individuals. The small ε term ensures numerical stability and prevents division by zero.

5. Genetic Operators

The following mechanisms are implemented to maintain genetic diversity while guiding the search toward better solutions:

- **Crossover:** it selects a random subtree from each parent and swaps them. To maintain structural constraints, the algorithm retries the swap if the resulting offspring exceed the allowed maximum tree depth.
- **Mutation:** it introduces variability by randomly altering parts of the individual. Two types of mutation are used, selected uniformly:
 - **Subtree Mutation:** replaces a randomly chosen subtree with a newly generated random subtree. The probability of performing the mutation increases with the depth of the node, ensuring deeper parts of the tree are more likely to be modified—thus increasing structural diversity where it matters most.
 - **Point Mutation:** alters a single node by either changing an operator (while preserving its arity), replacing a variable, or tweaking a constant. Like subtree mutation, the probability increases with depth, allowing fine-grained adjustments to evolve expressions incrementally.
- **Elitism:** the top $E\%$ of individuals (by fitness) are carried unchanged into the next generation to preserve the best solutions.

Each genetic operation ensures that the maximum depth constraint is respected.

6. Termination Criteria

The algorithm runs for a fixed number of generations G or until a solution reaches a predefined MSE threshold given as argument (early stopping).

At the end of the generations, the population is saved to disk (pickle) to allow warm-starting on future runs.

Experimental Setup

To evaluate our symbolic regression framework, we applied it independently to each dataset in the form `problem_i.npz`. For each dataset, we configured three key hyperparameters:

- **Population Size (P_i):** number of candidate solutions per generation.
- **Number of Generations (G_i):** maximum number of evolutionary cycles.
- **Maximum Tree Depth (D_i):** depth limit for generated expression trees to control model complexity and overfitting.

These hyperparameters were **empirically tuned** for each problem to strike a balance between computational cost and the quality of the resulting expressions.

Results

Below are the final symbolic expressions generated for each target function, along with their corresponding Mean Squared Error (MSE).

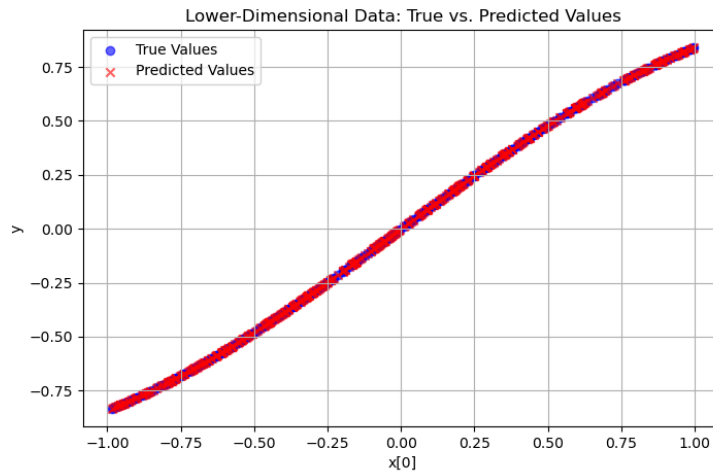
To better visualize model performance, we also include 3D scatter plots comparing the true vs. predicted values. Each plot represents the output distribution over pairs of input dimensions.

Function f1

- **MSE:** 0

- **Expression:**

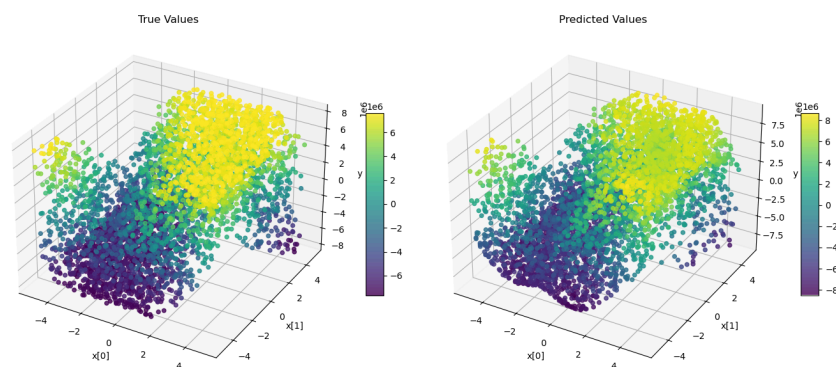
```
np.sin(x[0])
```

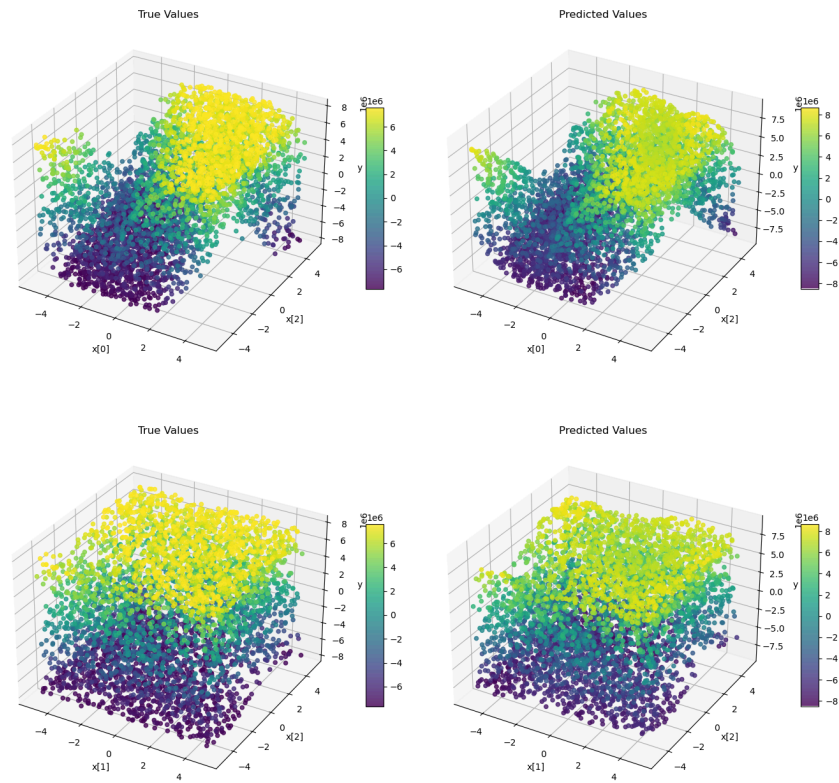


Function f2

- **MSE:** 2.241×10^{12}
- **Expression:**

```
(((((17.976 + (13.556 + 98.991)) + ((34.785 + np.cos(x[2])) - (x[0] * (x[0] + x[0])))) * (((7.68 + np.cos(x[2])) + (x[2] + 98.991)) + (((x[0] * x[2]) - (x[0] * x[1])) - (x[0] * (x[1] + x[0]))))) * (((((x[1] * x[2]) - np.exp(x[2])) + (98.991 + 84.959)) + ((90.002 / x[0]) * (x[1] + x[2]))) + (((11.47 * x[2]) + (x[1] + x[0])) + ((x[1] + x[0]) + (x[1] + x[0])))) * (((x[0] + x[0]) / (x[0] * 5)) - (x[1] + x[0])))) * ((np.tanh(((x[0] - (x[0] - x[1])) + 6)) * x[0]) / (((np.exp(np.sin(x[2])) + np.abs((x[1] + x[0]))) + (((x[0] - x[1]) + np.exp(x[1])) - np.tanh((x[0] / x[2]))) * (((x[2] + x[2]) + (x[1] + x[0])) * x[2]) / (((x[2] - 34.785) + (x[2] + 350)) + (np.abs(x[0]) * (x[0] * x[1]))))))))
```

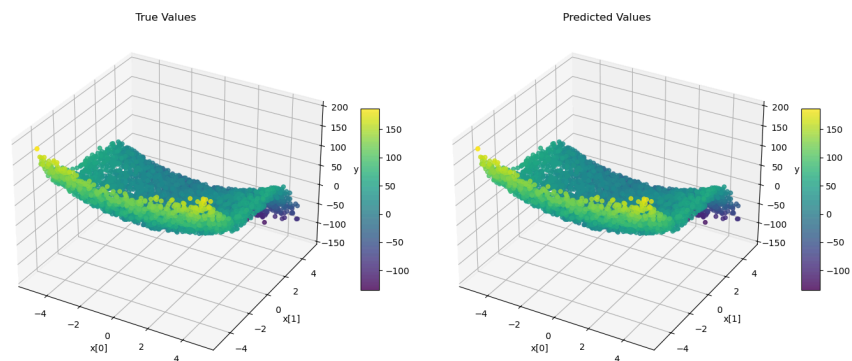


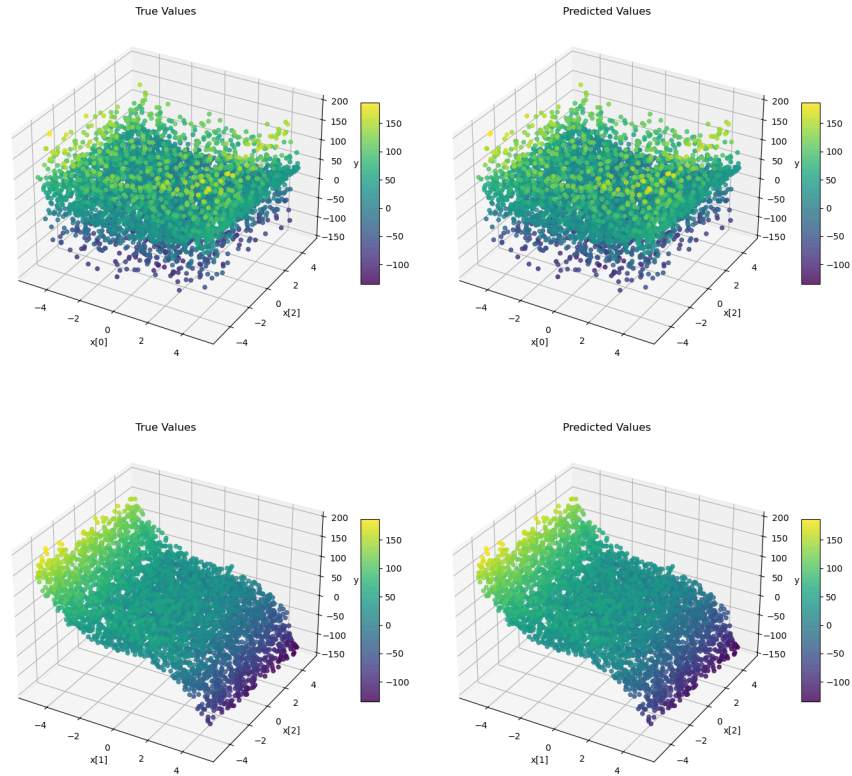


Function f3

- **MSE:** 3.31370×10^{-29}
- **Expression:**

$$((((x[0] * x[0]) + (x[0] * x[0])) + 4) - (((x[2] * 14) / 4) + (x[1] * (x[1] * x[1]))))$$

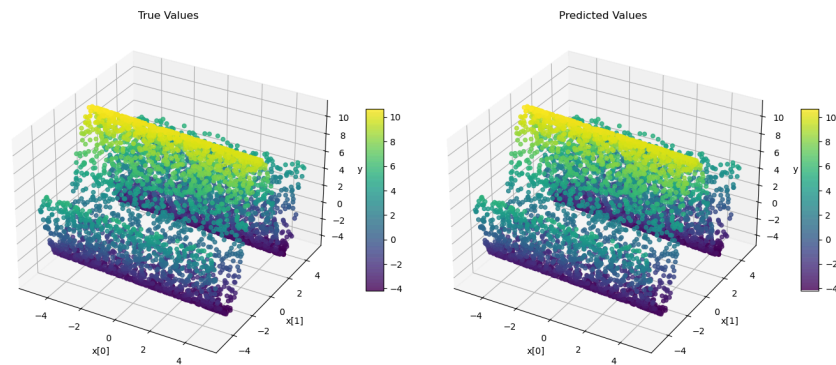




Function f4

- **MSE:** 2.13754×10^{-05}
- **Expression:**

```
(np.tan((((x[0] / -7.911) * np.cos(-13.373)) - np.tan(np.exp(-5.089)))) + (((np.cos(x[1]) + np.cos(-13.44)) * 7) + np.tan(-4.021)))
```

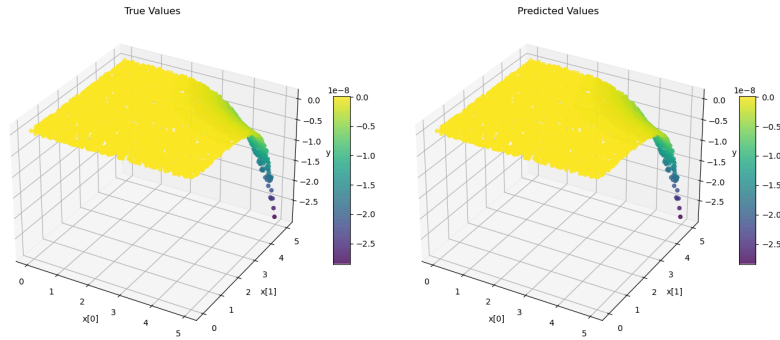


Function f5

- **MSE:** 1.91664×10^{-22}
- **Expression:**

```
2.8520706810421615e-10 * (((x[0] ** x[1]) - (((np.tan(-39.724) - np.cos((-20.187 + np.log(x[0])))) / 4.38) - np.log(x[1])) - (-18.065 - np.log(np.abs(np.abs((np.sin(-18.339) * np.sin(x[1])))))))) / (4.484 + (np.abs(np.log
```

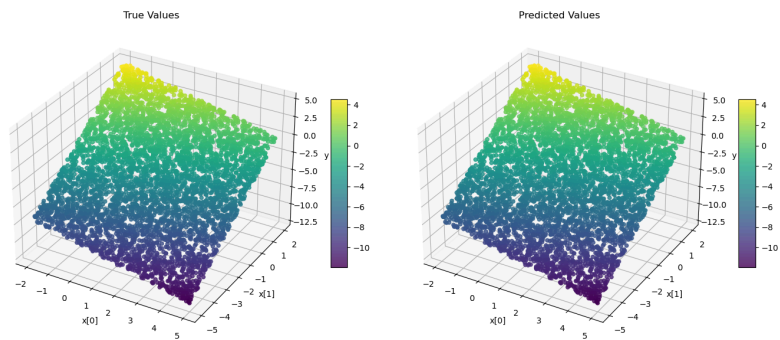
((19.817)) + -36.106))))



Function f6

- **MSE:** 5.73586×10^{-5}
- **Expression:**

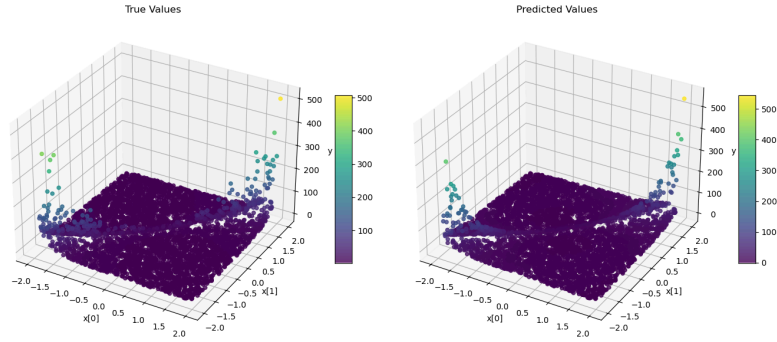
$((x[1] + x[1]) - (x[0] + ((x[0] / (\cos(\tan(-3.547)) - \tan(\tan(\tan(20.446)))) - (x[1] / (\cos((-12.357 + \tan(\tan(20.446)))) - \tan(\tan(\tan(20.446))))))))$



Function f7

- **MSE:** 5.02807×10^1
- **Expression:**

$(((((x[1] + ((10 / 92.147) + \sin(x[1]))) + (x[1] / (\tanh(x[0]) / x[1]))) + (\sin((x[1] - (x[0] - x[1]))) + (\sin(\sin(x[0]) + (\sin(x[0]) + (x[1] + x[1])))) * (x[0] / \cos(\cos((x[1] - x[0])))) + \exp(((2 + ((x[0] * x[1]) + ((9 - x[0]) / (20.038 + x[0])))) * \cos(((200 / (24.951 - 60.469)) * \tanh((x[0] - x[1]))))))))$

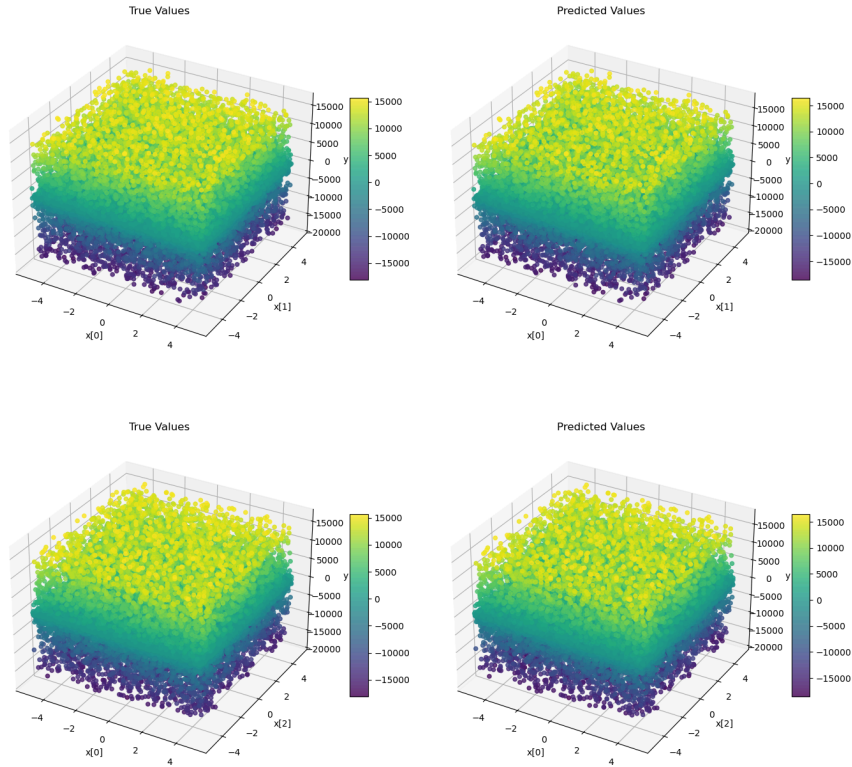


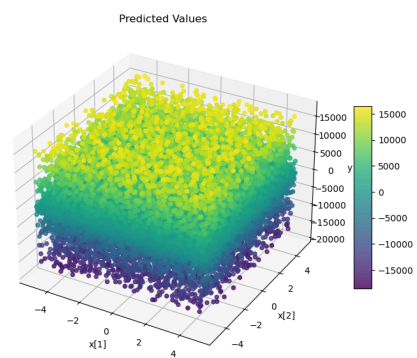
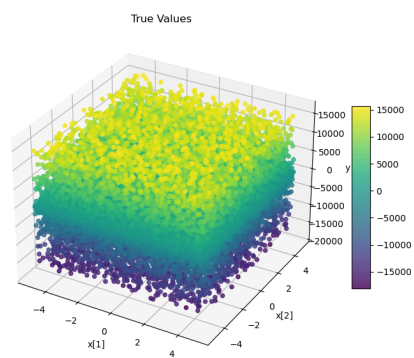
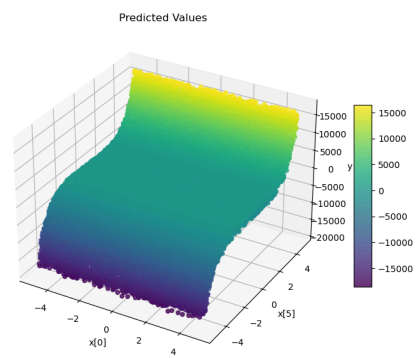
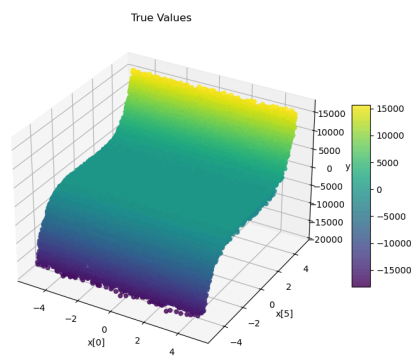
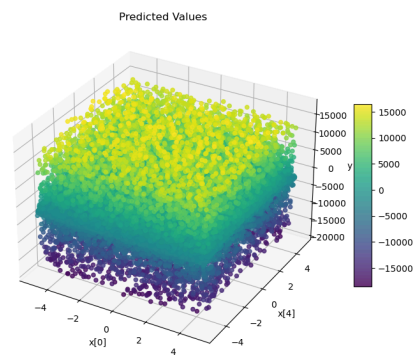
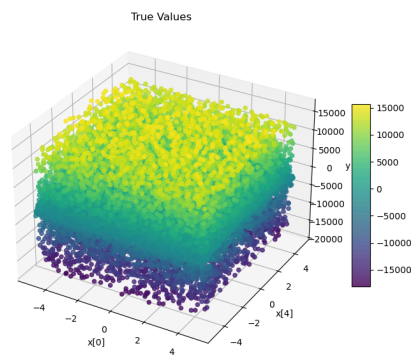
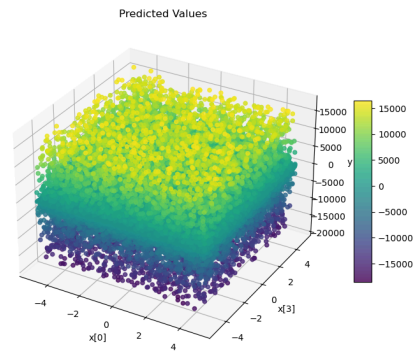
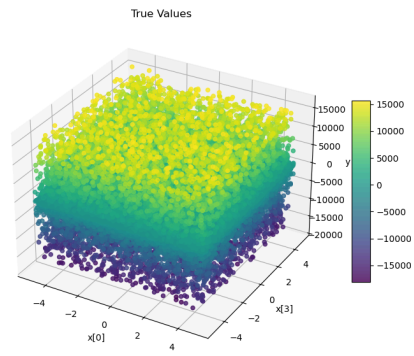
Function f8

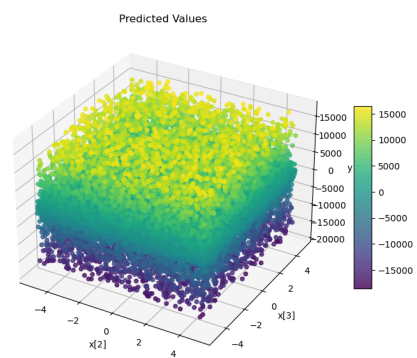
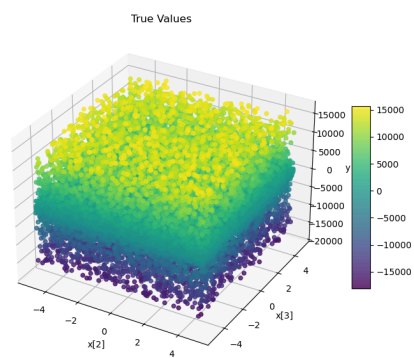
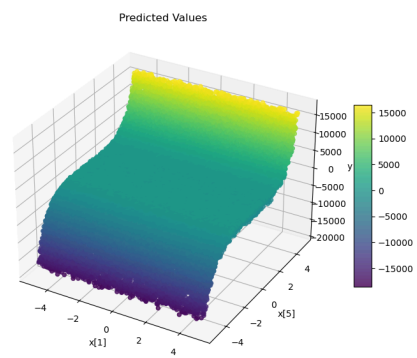
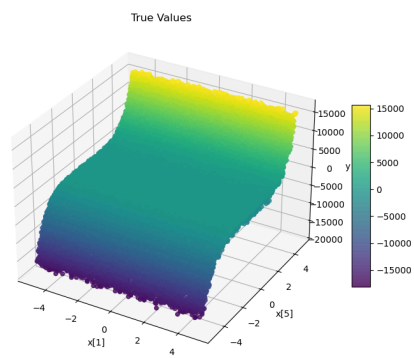
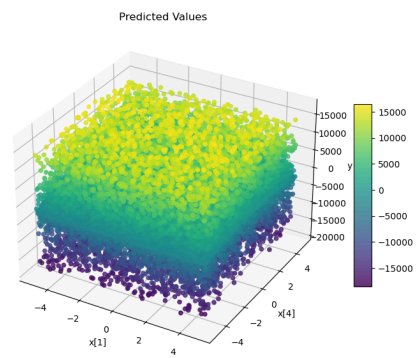
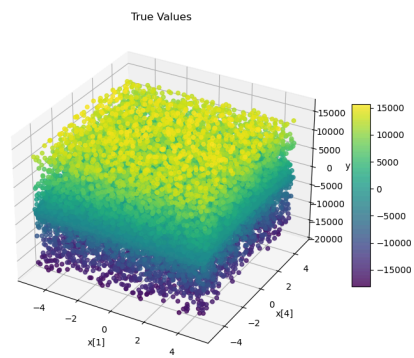
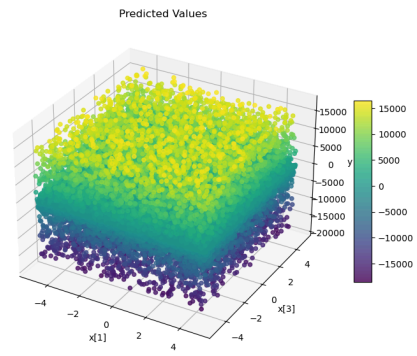
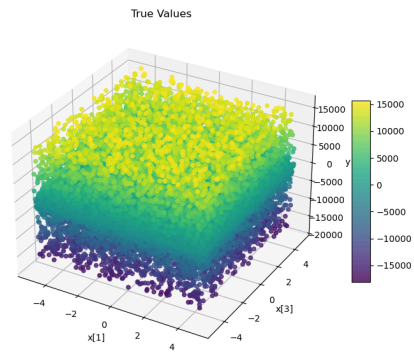
• **MSE:** 2.33539×10^4

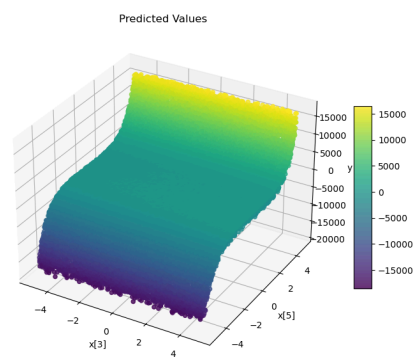
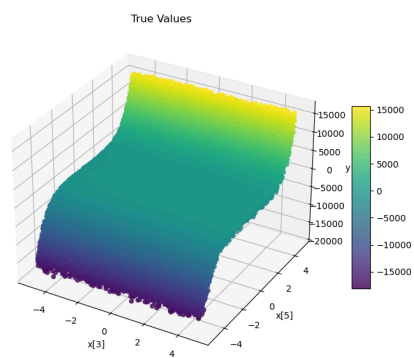
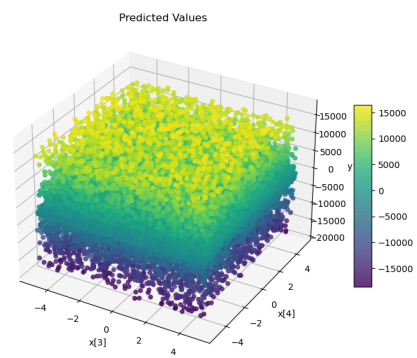
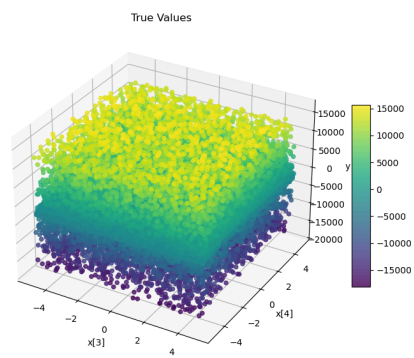
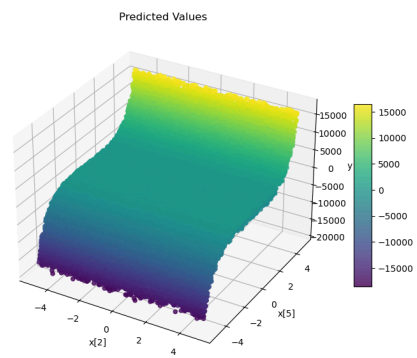
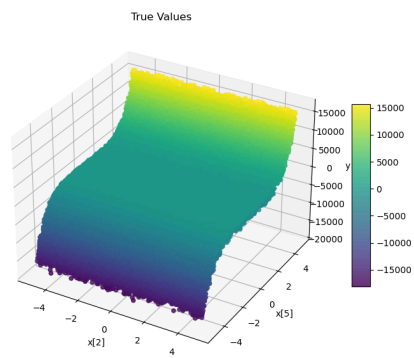
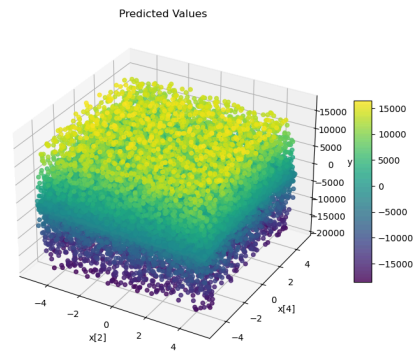
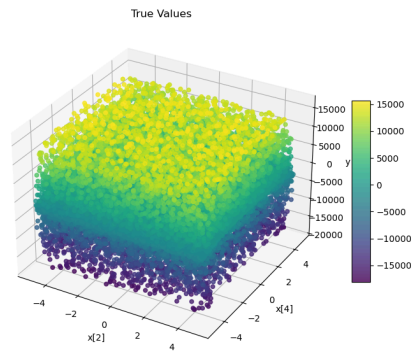
• **Expression:**

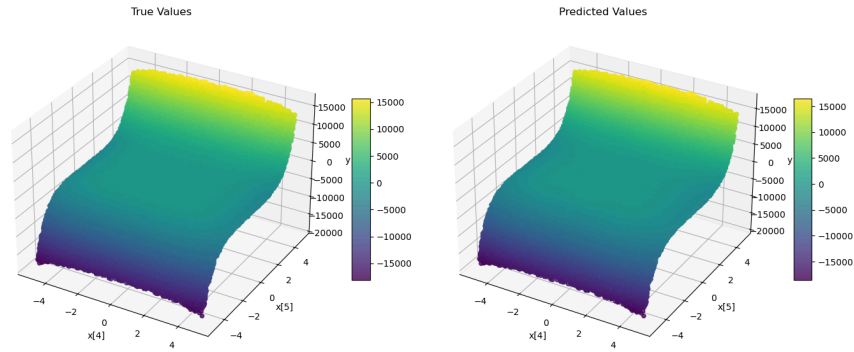
```
((np.exp(x[3]) - (((36.419 - (((-48.79 + -40.34) - (((33.294 + (36.419 - x[5])) / np.exp(np.cos(x[5])))) ** np.cos(x[4])))) * (np.cos(np.sin(np.exp((x[4] + -40.34)))) - (np.sin((x[4] + (-11.033 + x[4])) - (np.abs(np.cos((x[5] - 27.598))) + np.sin(x[5])))))))) + (((x[4] + np.sin(x[3])) - ((np.sin((np.exp(-1.663) - (x[4] + x[4])) - ((x[4] + x[4]) + x[4])) * (-1.663 - (x[4] + (x[4] + x[4]))))) + (np.exp(np.tan(np.sin(x[3])) - ((x[3] - 41.92) * (np.cos(x[4]) - x[3])))) - ((np.sin((np.tan(np.cos(-1.663) - (x[4] + x[4])) - (x[4] + (np.cos(np.log(26.12)) + (x[4] + x[4])))) * ((np.sin((np.tan(np.cos(-1.663) - (x[4] + x[4])) - (x[4] + np.sin(43.067))) - x[4])))) - ((-48.79 * np.sin(x[5]) + x[5])) + (np.abs(-21.878) * (((np.abs(np.abs(33.294)) - ((np.exp(-1.663) - (x[4] + x[4])) * (np.cos(-1.663) - x[4])))) + -22.214) * np.exp(np.exp((np.exp((np.abs(-16.989) - ((np.sin(x[4]) - (x[4] + x[4])) * ((41.697 ** -11.033) - (x[4] + x[4]))))) * -26.183)))) + (np.exp(np.abs(x[5]) * x[5]))))
```











Laboratory 1 - Set Cover Problem

Repository: https://github.com/RonPlusSign/CI2024_lab1

The Jupyter Notebook presents a comparison between three algorithms developed to solve the Set Cover Problem.

Each algorithm's performance has been evaluated across different instances based on execution time and the quality of the solution (fitness value).

A higher fitness value (closer to zero) indicates a better solution, as it represents a lower cost to cover all items in the universe.

The following table summarizes the execution results:

Metric	Instance 1	Instance 2	Instance 3	Instance 4	Instance 5	Instance 6
Universe Size	100	1,000	10,000	100,000	100,000	100,000
Num Sets	10	100	1,000	10,000	10,000	10,000
Density	0.2	0.2	0.2	0.1	0.2	0.3
Execution Time Solution 1	240ms	405ms	2.513s	45m 32s	47m 45s	50m 20s
Best Fitness Solution 1	-271.544	-13193.718	-175425.567	-2566839.346	-2796323.840	-2813886.565
Execution Time Solution 2	210ms	183ms	253ms	21.577s	13.416s	8.322s
Best Fitness Solution 2	-271.544	-5967.996	-101371.377	-1525514.078	-1715952.2693	-1760075.686
Execution Time Solution 3	595ms	615ms	1.224s	13m 28s	18m 43s	15m 3s
Best Fitness Solution 3	-136.55 (invalid solution)	-6678.618	-133464.135	-47044103.225	-98889589.561	-153822327.883

Result comparison

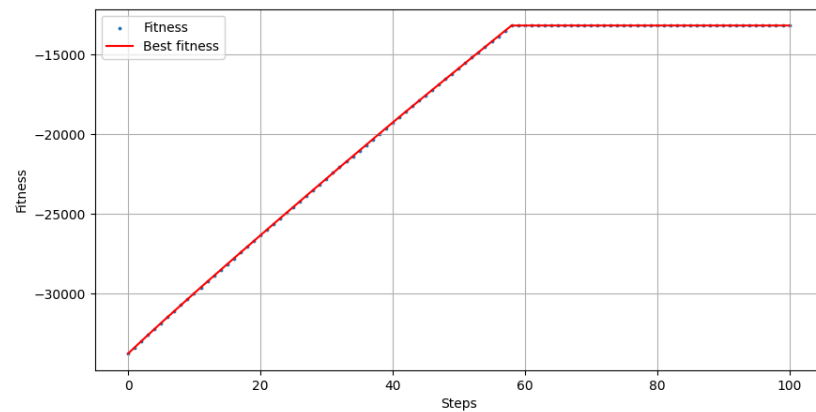
- **Solution 1 (Cost-Removal):** This algorithm is slower for larger instances but tends to yield reasonably good solutions with higher fitness values. However, it becomes computationally expensive as the problem size increases.
- **Solution 2 (Greedy Algorithm):** The fastest algorithm, that also yields the best fitness between all the 3 algorithms for all problem instances. It performs well in terms of both speed and solution quality.

- **Solution 3 (Simulated Annealing):** This approach is faster than solution 1 but slower than solution 2, and gives a solution quality that is worse than solution 1 and 2. Also, when we have a lower number of sets it can generate invalid solutions, as observed in instance 1.

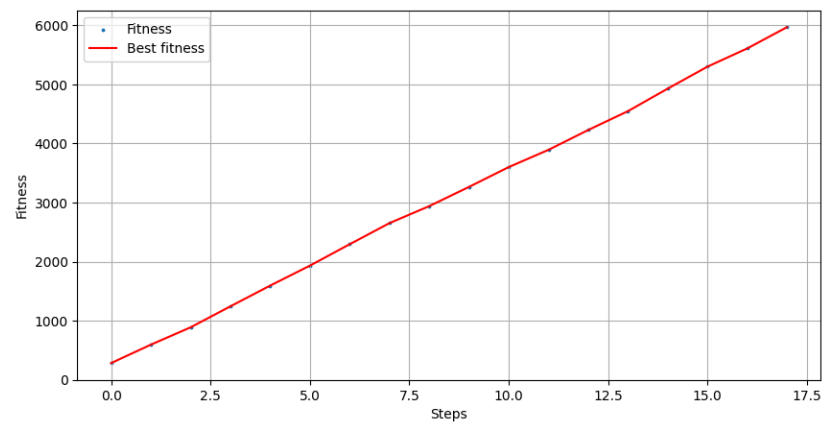
Plots

The following plots show the fitness values of the exploited solutions (blue dots) and the fitness of the best solution found by each algorithm for instance 2.

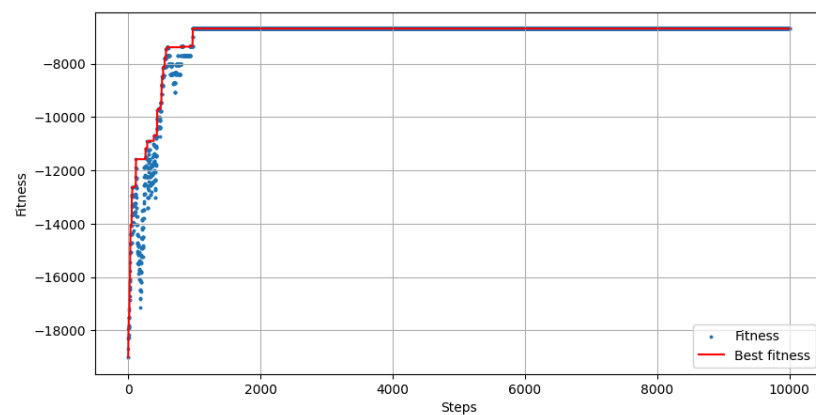
Solution 1



Solution 2



Solution 3



Suggestions

One of my colleagues suggested in a GitHub Issue ([link](#)) that the process of testing different instances can be automated.

Currently, to test different instances of the Set Cover problem, you need to manually change the Universe_size, Num_sets, and Density parameters. This process could be automated by using a loop to iterate over a predefined list of instances, such as the one below:

```
instances = [  
    {'Universe_size': 100, 'Num_sets': 10, 'Density': 0.2},  
    {'Universe_size': 1000, 'Num_sets': 100, 'Density': 0.2},  
    {'Universe_size': 10000, 'Num_sets': 1000, 'Density': 0.2},  
    {'Universe_size': 100000, 'Num_sets': 10000, 'Density': 0.1},  
    {'Universe_size': 100000, 'Num_sets': 10000, 'Density': 0.2},  
    {'Universe_size': 100000, 'Num_sets': 10000, 'Density': 0.3},  
]
```

An improvement could include writing a loop to automatically run the code for each instance without needing to manually change the values. This would make the notebook more efficient if the instances are only 5.

Laboratory 1 - Reviews of colleagues' works

In this section I report the reviews I made about my colleagues' laboratory 1. In this case I had to review the work of 2 colleagues: github.com/Lorkenzo and github.com/chiararcasale. Here I report the complete reviews and suggestions of improvements I created on GitHub.

Review 1 - https://github.com/Lorkenzo/CI2024_lab1/issues/1

Hi, congratulations for the great work of this lab. I'd like to make some suggestions on how to improve it.

First of all, the repo would be more clean if you didn't push all the files of the virtual environment, that should be kept local. I'd suggest you to create a `.gitignore` file to ignore the unnecessary files and remove them from the repo, keeping only the `README.md` and the `set-cover.ipynb` Notebook.

I really appreciate the fact that you plotted the cost variation depending on the number of iterations!

I see that almost half of the iterations are not increasing the fitness of the solution, so I'd suggest to lower the number of necessary iterations if you don't find an improving solution for too long (even though you stated in the readme that

"the number of iterations for each instance experiment is proportional to the size of the universe", in the last cases it's too many).

I noticed that you used a random initialization, and then you continued to add sets until you don't have a valid solution to start with:

```
solution = rng.random(NUM_SETS) < 0.75  
  
while not valid(solution):  
    solution[rng.integers(NUM_SETS)] = True
```

While this guarantees to start with a valid solution, I think it'd be more efficient to start directly with a solution composed by all sets (e.g. by using `solution = rng.random(NUM_SETS) < 1`), saving some iterations before starting the main loop.

I'd also suggest to try different values for the temperature's multiplying factors, other than `1.2`, and plot how its value changes during the algorithm execution.

I would also change the plots in order to show better the values near 0, because when you have a scale that goes from `1e8` to `0` it looks like the final cost is 0, even though it's not. You could use for example a logarithmic scale for the `y` axis.

Review 2 - https://github.com/chiararcasale/CI2024_lab1/issues/1

First of all, congratulations on the great work.

I really appreciated the fact that you compared 4 different algorithms and approaches to the problem, from the ones that perform the tweaks randomly (RMHCs & Simulated Annealing) following a black-box approach, to a greedy approach that exploits the problem definition.

Suggestions

I'd have loved to see some plots of the different algorithms that show how the cost of the solution changes based on the number of iterations, so that it'd be possible to know if there was a constant improvement or if the different approaches find the best solution in early stages, and then struggle to improve the solution.

As you mentioned in the README file, I think you should try to execute all the instances for all the provided algorithms, so that the comparison would be more accurate.

Comments to specific algorithms

Algorithms 1-2: RMHC

As you wrote in the notebook, you chose to use `solution = rng.random(NUM_SETS) < .2` as the starting solution, and change it until you find a valid solution.

While this approach guarantees that you always have a valid solution to start with, it's not efficient to do so because it might take a long time to find a valid solution randomly (by choosing only 20% of all possible sets), so this might result in poor performance in instances with a large amount of sets.

I think it'd be more efficient to use

`solution = rng.random(NUM_SETS) < 1` as the starting solution.

Algorithm 3: Simulated Annealing

Even though I don't have the cost/iteration graph, I also implemented a simulated annealing approach, trying different values for the initial temperature and cooling rate. I found out that the cooling rate of 0.99 is a bit too harsh and tends to make the temperature lower too fast. You should also try a higher value (for example I used 0.997) in order to favor exploration for more steps.

Algorithm 4: Greedy Algorithm

This algorithm is really smart and uses a clever set preselection.

One improvement I'd suggest is that the

`select_best_set(...)` method could use more built-in functions of numpy, such as `argmax`, in order to find the set with higher coverage-to-cost ratio.

An implementation example could be the following:

```
def select_best_set(current_solution, uncovered, NUM_SETS, SETS, COSTS):
    """Select the best set to add to the current solution to maximize coverage while minimizing cost"""

    mask = ~current_solution # Create a mask for sets not in the current solution
    coverage = np.sum(SETS[:, uncovered], axis=1) # Calculate the coverage for each set
    valid_coverage = coverage * mask # Mask out sets already in the current solution and those that cover no new elements

    scores = valid_coverage / COSTS # Calculate the scores for valid sets
    best_set = np.argmax(scores) # Find the index of the set with the highest score

    if scores[best_set] == 0:
        return None
    return best_set
```

Laboratory 2 - Traveling Salesman Problem (TSP)

Repository: https://github.com/RonPlusSign/CI2024_lab2

This project contains my two solutions for the Traveling Salesman Problem (TSP), and a final test section to evaluate their performance on different datasets (cities of different countries, from the `cities` folder).

For all the provided solutions, the total cost of the tour is calculated as the sum of the distances between consecutive cities. A plot of the final tour is also generated for visualization.

Solutions

I implemented two solutions for the TSP: a greedy algorithm and an Inver-Over crossover algorithm. Both solutions receive a distance matrix and a list of cities as input and return the best tour and its total cost.

Both approaches don't guarantee the optimal solution, but the Inver-Over algorithm is more accurate than the greedy algorithm, even though it is much slower.

To optimize the final tour in the test section, I first run the greedy algorithm to get a good initial solution, and then I use the inver-over algorithm to improve it again.

Solution 1: Greedy Algorithm

The greedy algorithm is a fast but approximate method for solving the TSP. It always chooses the nearest unvisited city until all cities are visited. The algorithm then returns to the starting city to complete the tour.

To execute it on different instances, I created the function `greedy_tsp` that receives a distance matrix and a list of cities and returns the best tour and its total cost.

Solution 2: Inver-Over Crossover Algorithm

The Inver-Over algorithm is a heuristic approach that combines inversion and crossover operations to evolve a population of solutions. It is slower but more accurate than the greedy algorithm.

The function

`inver_over` receives an initial TSP tour, the distance matrix, and some optional parameters (population size, number of generations, and mutation probability). It returns the best tour and its total cost.

I chose the hyperparameters of the Inver-Over algorithm by executing it with different values and selecting the best ones.

Tests

The final test block evaluates both solutions on the cities of all countries in the `cities` folder. It loads the cities, calculates the distance matrix, and runs both the greedy and inver-over algorithms. The results are logged for comparison.

Test Script

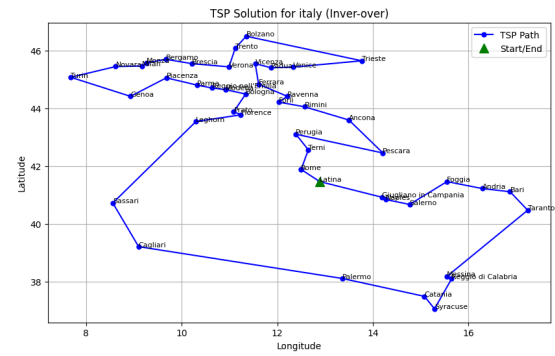
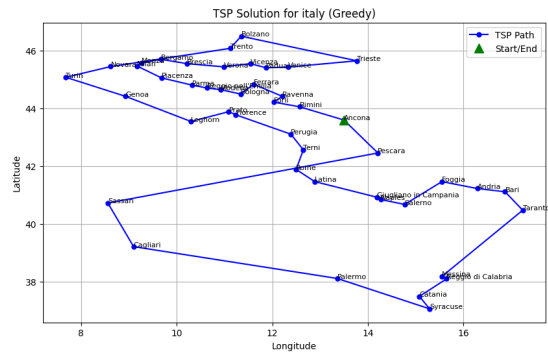
```
for file_name in os.listdir('cities'):
    logging.info(f"Processing {file_name}")
    # ...Load cities and compute distance matrix...
    # ...Run greedy algorithm...
    # ...Run inver-over algorithm...
    # ...Plot the final tours...
```

Visualization

Italy

Greedy: Found a path of 46 steps, total length
4436.03km

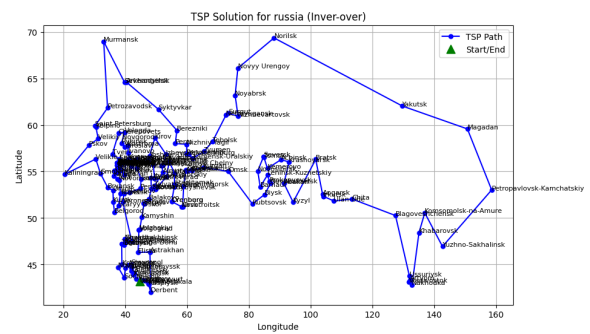
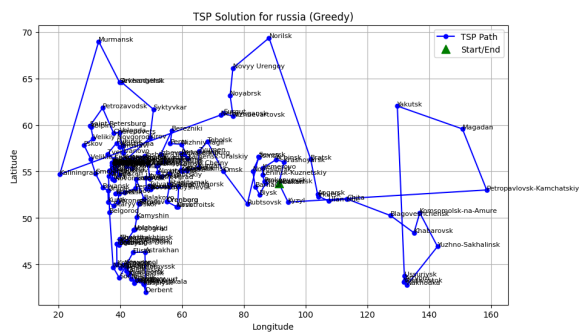
Inver-over: Found a path of 47 steps, total length
4175.23km



Russia

Greedy: Found a path of 167 steps, total length 42334.16km

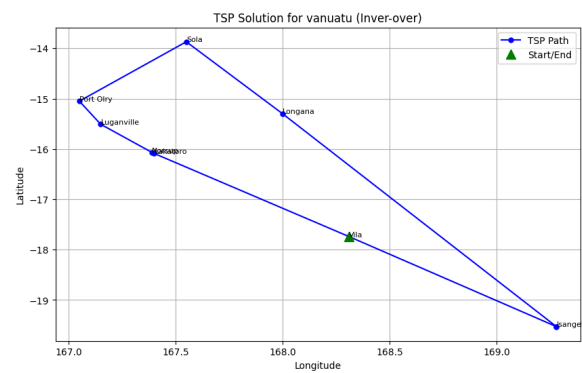
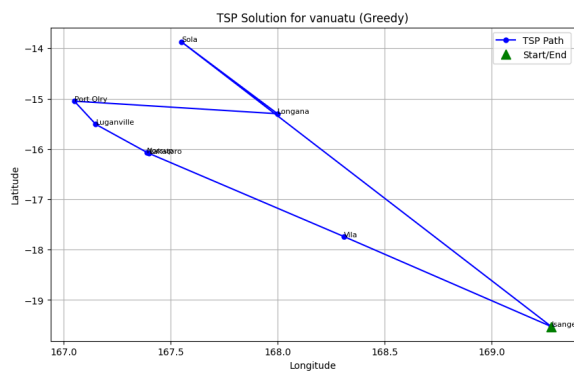
Inver-over: Found a path of 168 steps, total length 33602.44km



Vanuatu

Greedy: Found a path of 8 steps, total length 1475.53km

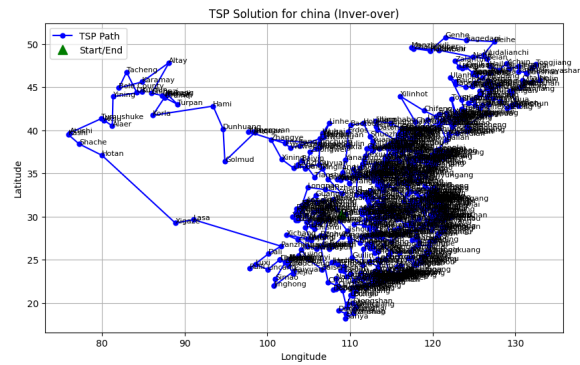
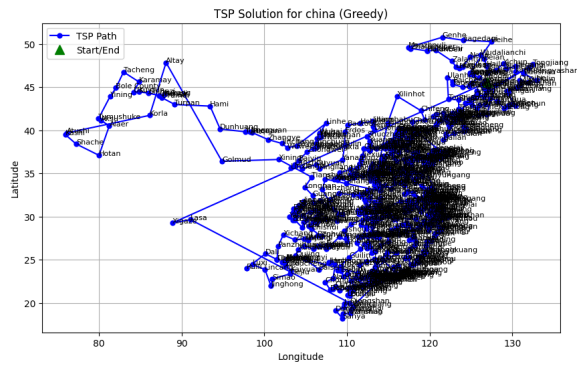
Inver-over: Found a path of 9 steps, total length 1345.54km



China

Greedy: Found a path of 726 steps, total length 63962.92km

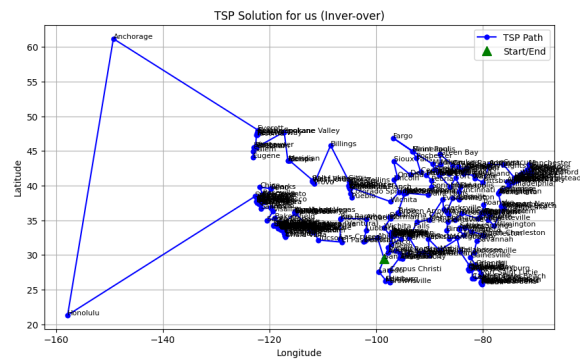
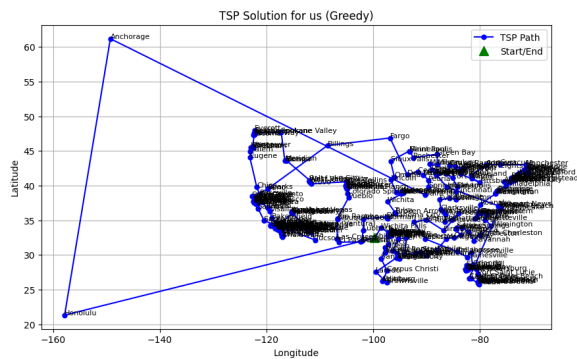
Inver-over: Found a path of 727 steps, total length 63925.54km



United States

Greedy: Found a path of 326 steps, total length 48050.03km

Inver-over: Found a path of 327 steps, total length 41279.01km



Suggestions

One of my colleagues suggested in a GitHub Issue ([link](#)) some improvements on the implementation.

There are some areas where the code could be improved. For example, the initialization process for the algorithms could be optimized. The Inver-Over algorithm, in particular, could benefit from starting with more diverse or optimized initial solutions to improve performance. Additionally, calculating the distance matrix for every pair of cities can be slow, especially with large datasets. Optimizing this step, perhaps by parallelizing it or caching the matrix, could reduce runtime significantly.

The Inver-Over algorithm could be made more flexible by adapting its parameters, such as population size and mutation probability. Also another good thing would be maybe having some results presented in the readme file swell as the description of the code and methodology. However, in general, the code presents a very well solution to the given task and with these few minor tweaks, it could be perfected a little bit.

Laboratory 2 - Reviews of colleagues' works

In this rection I report the reviews I made about my colleagues' laboratory 2. In this case I had to review the work of 2 colleagues: github.com/SaraaFrancee and github.com/criciky. Here I report the complete reviews and suggestions of improvements I created on GitHub.

Review 1 - https://github.com/SaraaFrancee/CI2024_lab2/issues/3

Hi there, I'm Andrea. First of all, you did a great job in this lab! I have a few suggestions to improve your notebook.

Comments & Suggestions

The performances you obtained are quite good, even when compared to the solutions I obtained with my greedy algorithm (nearest city first).

I liked the choice to use the results of the greedy algorithms as a starting point for the simulated annealing approach, so that you already start from a valid (but not optimal) solution.

Consider using an evolutionary algorithm like the ones presented during the lectures (e.g. **inver-over-crossover algorithm**) to optimize the calculation of distances between cities. This can improve the efficiency and accuracy of the results, and it's a different approach from the one we did in the previous laboratory (lab1).

Other improvements you can consider:

- visualize the path between cities using matplotlib, so that you have a visual understanding if the path is feasible and makes sense, or is similar to random
- create a for loop to test all instances one after the other, instead of specifying just one of the countries and changing it manually

Review 2 - https://github.com/crickey/C12024_lab2/issues/1

Hi there, I'm Andrea. First of all, you did a great job in this lab! I have a few suggestions to improve your notebook.

Comments & Suggestions

As far as I understand, the `greedy_heuristic` function creates a certain number of paths, half using a nearest neighbor approach with different starting cities, and the second half are random paths.

This is a good starting point for the inver-over-crossover algorithm below, but the greedy approach (which is deterministic) will return many similar or equal paths, even with different starting cities.

I'd suggest you to change this approach adding some randomness in the population generation process, in order to favor *exploration*.

I tested the distance dataframe generation `dist_df` and I found a bug: the latitude and longitude are not obtained using `c.lat` and `c.lon`, you should use `c.x` and `c.y` instead.

```
dist_df = np.zeros((cities.shape[0], cities.shape[0]))
for c1, c2 in itertools.combinations(cities.itertuples(), 2):
    distance = geodesic((c1.x, c1.y), (c2.x, c2.y)).km
    dist_df[c1.Index, c2.Index] = distance
    dist_df[c2.Index, c1.Index] = distance
dist_df
```

The `inver_over_crossover` algorithm is correctly implemented, but there's no specification about how you chose the hyperparameters (population_size = 100, mutation_rate = 0.01, generations = 1000). I'd suggest to test different values and check how the performance changes.

It looks like you inserted a classic Python script in your Notebook, because you also inserted unnecessary code such as `if __name__ == '__main__':`, and everything is in one single python block.

I'd be better to split it in multiple blocks: one for the

`greedy_heuristic`, one for the `evolutionary_algorithm`, and one for testing it (your `main` code).

I saw you have an error `KeyError` in the notebook you uploaded, but re-running it gave me no error.

The solution I obtain using your algorithm for Italy has the same performance as the greedy algorithm I implemented: 4436.03km.

Check if you're changing anything from the basic greedy solution, because my version of inver-over-crossover (check my repo for details) has an improvement in terms of final performance (4175.23km as shortest path).

Other improvements you can consider:

- visualize the path between cities using **matplotlib**, so that you have a visual understanding if the path is feasible and makes sense, or is similar to random

- create a for loop to test all instances one after the other, instead of specifying just one of the countries and changing it manually

Laboratory 3 - n-puzzle problem

Repository: https://github.com/RonPlusSign/CI2024_lab3

This project contains my 3 solutions for the n-puzzle problem, which is a puzzle game consisting of a board of $n \times n$ cells, where n is an integer and $n > 2$. The board contains $n^2 - 1$ tiles numbered from 1 to $n^2 - 1$, and one empty cell (represented by the value 0 in this project).

The goal of the game is to rearrange the tiles in the board to reach a goal configuration, by moving one of the adjacent tiles into the empty cell.

The goal is a configuration where the tiles are ordered from 1 to $n^2 - 1$ in row-major order, and the empty cell is in the bottom-right corner of the board.

The project contains 3 solutions for the n-puzzle:

1. **Depth First Search (DFS)**
2. **Breadth First Search (BFS)**
3. **A* Search Algorithm**

The initial puzzle configuration is generated by shuffling the goal configuration with 100000 valid steps.

Solution 1: Depth First Search (DFS)

Depth First Search (DFS) is an algorithm for traversing or searching tree or graph data structures.

The algorithm starts at the root node (in this case the initial puzzle configuration) and explores as far as possible along each branch before backtracking.

The version of DFS I implemented has a **fixed maximum depth**, in order to avoid non-termination, cycles and memory overflow.

Algorithm steps:

1. Begin the search at the initial puzzle configuration and push it onto a stack
2. Pop the top node from the stack, mark it as visited, and process it (check if it is the goal, if not, generate its children)
3. Push all unvisited adjacent nodes of the current node onto the stack (puzzle configurations that can be reached from the current configuration by moving a tile)
4. Repeat steps 2 and 3 until the stack is empty or the goal is found.

DFS has been implemented using a *stack* data structure, and can be done either iteratively (as I did) or via recursion.

Each element of the stack is composed by 3 elements:

- The current puzzle configuration
- The history of how the current configuration was reached
- The depth of the current configuration (used to restrict the search up to a maximum depth)

Issues of DFS:

- DFS may not find the shortest path to the goal node, because a maximum depth of the tree is reached before backtracking
- Increasing the maximum depth of the tree may lead to a memory overflow or non-termination
- The computational cost is higher when compared to the other 2 solutions

Note: sometimes this algorithm fails to find a solution even for 3×3 puzzles, and thus it cannot be considered a good solution.

Solution 2: Breadth First Search (BFS)

Breadth First Search (BFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (in this case the initial puzzle configuration) and explores all neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

Algorithm steps:

1. Begin the search at the initial puzzle configuration and push it onto a queue
2. Pop the front node from the queue, mark it as visited, and process it (check if it is the goal, if not, generate its children)
3. Push all unvisited adjacent nodes of the current node onto the queue (puzzle configurations that can be reached from the current configuration by moving a tile)
4. Repeat steps 2 and 3 until the queue is empty or the goal is found.

BFS has been implemented using a *queue* data structure.

Each element of the queue is composed by 2 elements:

- The current puzzle configuration
- The history of how the current configuration was reached

Issues of BFS:

- High memory usage, because it stores all the nodes at the current depth level
- The computational cost is higher when compared to the A* algorithm
- The algorithm may take a long time to find a solution for large puzzles

Note: this algorithm fails to find a solution for 4×4 puzzles because it finishes all the available memory of a 32GB RAM machine.

For 3×3 puzzles, the algorithm takes a few seconds to find a solution.

Solution 3: A* Search Algorithm

A* is a graph traversal and path search algorithm, which is widely used in many fields of computer science due to its completeness, optimality, and optimal efficiency.

The algorithm uses a best-first search and finds the least-cost path from the initial node to the goal node.

The A* algorithm evaluates nodes by combining the cost to reach the node (g) and the cost to get from the node to the goal (h).

Solution 3: A* Algorithm

The A* algorithm is a search algorithm designed to solve optimization problems like the n-puzzle efficiently. It uses a combination of the **path cost** (g, the number of moves so far) and a **heuristic estimate** (h, the estimated moves to reach the goal) to prioritize exploration. This ensures an optimal solution if the puzzle is solvable.

Algorithm steps:

- Initialize the priority queue with the starting configuration (state) as a tuple, a path containing just the starting state, and a path cost $g = 0$.
- While the priority queue is not empty:
 1. Pop the state with the lowest $f = g + h$.

2. If the state is the goal (`is_goal`), return the `path` to the solution.
3. If the state was already visited, skip it. Otherwise, mark it as visited.
4. For each valid action, generate a new state by moving a tile:
 - Calculate the new path cost `g` and heuristic `h`
 - Add the new state to the priority queue along with its updated cost and path history

Each element in the queue contains:

- The priority `f = g + h`
- A tiebreaker to prioritize states with the same priority (FIFO)
- The current puzzle configuration
- The history leading to the current state
- The path cost `g` to reach the current state

Heuristic:

The **Manhattan Distance** heuristic is used, which calculates the sum of the distances of each tile from its goal position. This heuristic is admissible (never overestimates) and ensures the A* algorithm finds an optimal solution.

Advantages of A*:

- Guarantees the shortest path to the goal configuration
- Uses an admissible heuristic to prioritize states closer to the solution

Issues of A*:

- Memory-intensive: Stores all visited and queued states, leading to high memory usage
- Computationally expensive for larger puzzles (`n > 4`) due to exponential growth of states

Note: The A* algorithm performs well for puzzles up to 4×4, solving them optimally and efficiently. However, for larger puzzles (5×5 or greater), its memory and computational requirements make it impractical without additional optimizations.

Suggestions

One of my colleagues suggested in a GitHub Issue ([link](#)) some improvements on the implementation.

1. It would be helpful to include one `requirements.txt` file so that others can easily replicate the environment and run the code without issues and also ensure that dependencies are clearly outlined for anyone running the project.
2. It would be helpful to include the execution time, to see how long each implementation (DFS, BFS, A*) took to solve puzzles of different dimensions, like 3×3, 4×4, and so on.
3. Introduce a penalty system to make the heuristic more effective, to help speed up the solution for bigger puzzles, like 4×4 or even larger.

Laboratory 3 - Reviews of colleague's works

In this rection I report the reviews I made about my colleague's laboratory 3. In this case I had to review the work of one colleague: github.com/GiorgiaModi. Here I report the complete review and suggestions of improvements I created on GitHub.

Review 1 - https://github.com/GiorgiaModi/CI2024_lab3/issues/1

Hi there, I'm Andrea. First of all, you did a great job in this lab! I found the README file really useful to understand the algorithms you implemented, as well as the tables containing the results in terms of execution time and performance.

Comments & suggestions

I have a few suggestions to improve your notebook.

1. I saw that in some cases your DFS algorithm doesn't find a solution. In these cases, I think it should be worth trying to increase the depth. E.g. for the 4×4 puzzle with 10k randomization timesteps and a maximum depth of 20 your algorithm fails to find a valid solution.
2. You might also consider implement another famous algorithm: Breadth First Search (BFS), which is similar to the DFS you implemented. You just need to pop items from the front of the queue, and this can be performed by changing from `stack.pop()` to `stack.popleft()` in the DFS implementation (in this case a better suiting name might be `queue`, instead of `stack`)

```
# ...  
while stack:  
    current_state, path, depth = stack.popleft()  
# ...
```

3. Include some plots on how the metrics changes during the problem solving, in order to check the convergence speed towards the solution (and also to visualize if the DFS algorithm stops near the solution or not)